

STATISTICS

for Machine Learning, Deep Learning, AI & Data Science

Beginner-Friendly • Intuition-Driven • 60+ Pages

From data types and distributions to EDA, regression, and real ML applications
Python examples with NumPy, Pandas, Matplotlib, Seaborn, SciPy & Scikit-learn

Author: Your Name | Date: 2025 | Version 1.0

Designed for ML students, data scientists, and AI practitioners

Table of Contents

Table of Contents	2
Introduction to Statistics for ML & AI	5
What is Statistics?	5
Why Statistics is Critical in Machine Learning & Data Science	5
Descriptive vs Inferential Statistics	6
The Data Science Workflow	6
Chapter 1: Types of Data.....	7
1.1 Why Data Types Matter	7
1.2 Qualitative vs Quantitative	7
1.3 Discrete vs Continuous.....	7
1.4 Levels of Measurement: NOIR	7
1.5 Python: Exploring Data Types	8
Chapter 2: Data Representation and Visualisation.....	10
2.1 Frequency Distributions.....	10
2.2 Histograms — The Most Important Plot in Statistics.....	10
2.3 Python: Complete Visualisation Suite	10
Chapter 3: Measures of Central Tendency	13
3.1 The Three Measures	13
The Mean — The Arithmetic Average.....	13
The Median — The Middle Value	13
The Mode — The Most Frequent Value	13
3.2 Python: Central Tendency.....	14
Chapter 4: Measures of Dispersion.....	16
4.1 Why Spread Matters	16
Range	16
Variance — Average Squared Deviation	16
Standard Deviation — The Most Used Measure	16
Interquartile Range (IQR)	17
4.2 Python: Dispersion & Box Plots.....	17
Chapter 5: Shape of Data — Skewness and Kurtosis	19
5.1 Skewness — Asymmetry of Distribution.....	19
5.2 Kurtosis — Peakedness and Tail Weight.....	19
5.3 Python: Skewness, Kurtosis, and Transformations.....	20

Chapter 6: The Normal Distribution.....	22
6.1 What is the Normal Distribution?	22
6.2 Why ML Assumes Normality.....	22
6.3 The Z-Score — Standardising Values	23
6.4 Python: Normal Distribution.....	23
Chapter 7: Outliers and Data Scaling	25
7.1 What are Outliers?.....	25
Method 1: Z-Score Method.....	25
Method 2: IQR Method (More Robust)	25
7.2 Data Scaling and Normalisation	25
Min-Max Normalisation (0 to 1 scaling).....	25
Standardisation (Z-score scaling)	26
7.3 Python: Outlier Detection and Scaling.....	26
Chapter 8: Correlation and Covariance.....	29
8.1 Covariance — Direction of Relationship	29
8.2 Pearson Correlation — Standardised Covariance.....	29
8.3 Correlation vs Causation	29
8.4 Python: Correlation Analysis.....	30
Chapter 9: Sampling and Hypothesis Testing	32
9.1 Population vs Sample.....	32
Sampling Bias	32
9.2 Hypothesis Testing — The Basic Idea.....	32
The p-value — Plain English	33
9.3 Python: Hypothesis Testing	33
Chapter 10: Linear Regression	35
10.1 The Line of Best Fit.....	35
10.2 Residuals — The Errors	35
10.3 Model Evaluation Metrics.....	35
10.4 Python: Complete Regression.....	36
Chapter 11: Exploratory Data Analysis (EDA)	38
11.1 What is EDA and Why?	38
11.2 The EDA Toolkit.....	38
11.3 Python: Full EDA Workflow.....	38
Chapter 12: Bias-Variance Tradeoff.....	41
12.1 The Central Challenge of Machine Learning.....	41

12.2 Visualising the Tradeoff	41
Chapter 13: Statistics Powering ML Models	44
13.1 Linear Regression — A Statistical Foundation	44
13.2 Naive Bayes — Pure Probability.....	44
13.3 k-Means Clustering — Statistics in Unsupervised Learning.....	44
13.4 Statistics in Neural Networks	44
13.5 Python: Statistics for ML Evaluation	45
Chapter 14: Complete Mini Project — End-to-End Data Analysis	47
Project: Predicting Diabetes Risk from Health Metrics.....	47
Chapter 15: Summary and Further Reading	52
15.1 The Statistics-ML Connection: Full Picture	52
15.2 Recommended Next Steps.....	52
Courses and Books	52
Practice Projects.....	52
15.3 Final Comprehensive Quiz	53

Introduction to Statistics for ML & AI

What is Statistics?

Statistics is the science of collecting, organising, analysing, interpreting, and presenting data. In its simplest form, statistics helps us make sense of large amounts of information by summarising it into meaningful numbers and patterns.

Think of statistics as the bridge between raw data and knowledge. You might have a spreadsheet with 10 million rows — statistics gives you tools to understand it, question it, and draw reliable conclusions from it.

Statistics = The Language of Data

Without statistics, data is just noise. With statistics, data becomes insight.

A machine learning model is essentially a statistical model: it looks at data patterns, makes assumptions about distributions, estimates parameters, and quantifies uncertainty.

Every single ML algorithm — from linear regression to deep neural networks — is built on statistical foundations.

Why Statistics is Critical in Machine Learning & Data Science

Statistics Concept	ML/AI Application
Mean, Variance	Feature scaling, understanding data spread
Normal Distribution	Assumptions in regression, weight initialisation in NNs
Correlation	Feature selection, detecting multicollinearity
Hypothesis Testing	A/B testing, model comparison, significance of results
Probability	Bayesian models, probabilistic classifiers (Naive Bayes)
Regression	Predicting continuous outcomes, model interpretability
Bias-Variance Tradeoff	Overfitting vs underfitting — the central ML challenge
Outlier Detection	Data cleaning, anomaly detection in production
Confidence Intervals	Model evaluation, reliability of predictions
Sampling	Train/test split, cross-validation, bootstrapping

Descriptive vs Inferential Statistics

Type	Purpose	Examples	ML Use
Descriptive	Summarise what IS in the data	Mean, median, std, histogram	EDA, data quality checks
Inferential	Draw conclusions beyond the data	Hypothesis tests, confidence intervals	Model evaluation, A/B testing

Real-World Analogy

Descriptive statistics is like looking at a photo album and describing what you see: 'Most photos are from summer, the average smile rating is 8/10.' Inferential statistics is drawing conclusions: 'Based on these 100 photos, people in our family are probably happy people.' — you're making a claim about a larger truth from limited evidence.

The Data Science Workflow

1. Collect data
2. Clean and preprocess (statistics helps detect issues)
3. Explore with EDA (descriptive statistics)
4. Model (uses probability and statistical assumptions)
5. Evaluate (inferential statistics, metrics)
6. Deploy and monitor (statistics tracks drift and degradation)

Chapter 1: Types of Data

1.1 Why Data Types Matter

Not all data is the same. The type of data you have determines which statistical methods and ML algorithms you can apply. Using the wrong method for the wrong data type is one of the most common beginner mistakes.

1.2 Qualitative vs Quantitative

Category	Description	Examples	ML Treatment
Qualitative (Categorical)	Labels, names, groups — no natural numeric meaning	Colour, gender, country, species	One-hot encoding, label encoding
Quantitative (Numerical)	Numbers that represent measurable quantities	Age, price, temperature, score	Use directly, may need scaling

1.3 Discrete vs Continuous

Type	Description	Examples	ML Notes
Discrete	Countable, finite possible values	Number of children, dice roll, word count	Often treated as categorical or integer
Continuous	Can take any value in a range	Height, weight, time, temperature	Normalisation/scaling usually needed

1.4 Levels of Measurement: NOIR

The NOIR framework (Nominal, Ordinal, Interval, Ratio) classifies data by the mathematical operations that make sense for it. Understanding this determines which statistics and models are valid.

Level	Description	Example	Valid Maths	ML Use
Nominal	Named categories — no order	Colours, blood types	Count, mode	One-hot encode
Ordinal	Ordered categories — gaps unequal	Rating: Poor/OK/Great	Median, mode, rank	Label encode carefully
Interval	Equal spacing — no true zero	Temperature °C, IQ score	Mean, std — ratios meaningless	Can normalise
Ratio	Equal spacing + true zero	Height, weight, price, age	All: mean, std, ratios	All operations valid

Why This Matters for Machine Learning

Nominal data: You CANNOT feed 'Red=1, Blue=2, Green=3' into a regression — the model will think Green > Blue > Red numerically. Use one-hot encoding instead.

Ordinal data: Order matters but gaps don't. 'Poor=0, Good=1, Excellent=2' is OK for tree models but risky for regression.

Ratio data: Most ML algorithms expect this. If your data isn't ratio-scale, pre-processing is needed.

1.5 Python: Exploring Data Types

```
import pandas as pd
import numpy as np

# Create a sample dataset
data = {
    'name':      ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'age':       [25, 32, 28, 45, 35],
    'salary':    [50000, 70000, 60000, 90000, 65000],
    'department': ['HR', 'Engineering', 'HR', 'Engineering', 'Marketing'],
    'rating':    ['Good', 'Excellent', 'Poor', 'Excellent', 'Good'],
    'score':     [72.5, 88.0, 61.3, 95.2, 79.8],
}
df = pd.DataFrame(data)

# Inspect data types
print('Data Types:')
print(df.dtypes)

# Separate numeric vs categorical
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()

print('\nNumeric columns: ', numeric_cols)
print('Categorical columns:', categorical_cols)

# Value counts for categorical
print('\nDepartment counts:')
print(df['department'].value_counts())

# Encode ordinal 'rating'
rating_map = {'Poor': 0, 'Good': 1, 'Excellent': 2}
df['rating_encoded'] = df['rating'].map(rating_map)
print('\nRating encoded:')
print(df[['rating', 'rating_encoded']])

# One-hot encode nominal 'department'
df_encoded = pd.get_dummies(df, columns=['department'], prefix='dept')
print('\nAfter one-hot encoding:')
```

```
print(df_encoded.head())
```

Mini Quiz

Q1: Is 'temperature in Celsius' nominal, ordinal, interval, or ratio? Why?

✓ *Answer: Interval. It has equal spacing (1°C difference is always the same) but no true zero — 0°C does not mean 'no temperature'. So ratios like '40°C is twice as hot as 20°C' are meaningless.*

Q2: Why should you NOT use label encoding (Red=0, Blue=1, Green=2) for nominal colours in a linear regression?

✓ *Answer: Regression assumes numeric relationships. It would interpret Blue as 1 unit more than Red, and Green as 2 units more. This is mathematically nonsensical for colours with no inherent order.*

Chapter 2: Data Representation and Visualisation

2.1 Frequency Distributions

A frequency distribution organises data by showing how often each value (or range of values) occurs. It is the first step in understanding the shape and spread of your data.

Frequency Distribution Example

Exam scores: [55, 60, 60, 65, 70, 70, 70, 75, 80, 85, 85, 90]

Score Range	Frequency	Relative Freq.
50-60	2	16.7%
61-70	4	33.3%
71-80	3	25.0%
81-90	3	25.0%

2.2 Histograms — The Most Important Plot in Statistics

A histogram shows the distribution of a continuous variable by dividing its range into bins and showing how many data points fall into each bin. Unlike a bar chart (which shows categories), a histogram shows distribution shape.

What to look for in a histogram:

- Centre: where is the bulk of the data?
- Spread: how wide is the distribution?
- Shape: symmetric (bell), skewed left, skewed right, bimodal?
- Outliers: isolated bars far from the main group

2.3 Python: Complete Visualisation Suite

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

# Simulate a dataset: customer ages and purchase amounts
n = 300
```

```

ages      = np.random.normal(35, 10, n).astype(int).clip(18, 70)
salaries  = np.random.normal(55000, 15000, n).clip(20000, 150000)
dept      = np.random.choice(['HR', 'Eng', 'Marketing', 'Sales'], n)

df = pd.DataFrame({'age': ages, 'salary': salaries, 'department': dept})

# — 1. Histogram —————
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(df['age'], bins=20, color='steelblue', edgecolor='white', alpha=0.8)
axes[0].set_title('Age Distribution (Histogram)')
axes[0].set_xlabel('Age'); axes[0].set_ylabel('Frequency')
axes[0].grid(axis='y', alpha=0.3)

axes[1].hist(df['salary'], bins=25, color='coral', edgecolor='white', alpha=0.8)
axes[1].set_title('Salary Distribution (Histogram)')
axes[1].set_xlabel('Salary ($)'); axes[1].set_ylabel('Frequency')
axes[1].grid(axis='y', alpha=0.3)

plt.tight_layout(); plt.savefig('histograms.png', dpi=150)
plt.show()

# — 2. Bar Chart (Categorical) —————
plt.figure(figsize=(8, 4))
dept_counts = df['department'].value_counts()
plt.bar(dept_counts.index, dept_counts.values,
        color=['#2196F3', '#4CAF50', '#FF9800', '#E91E63'])
plt.title('Employees per Department'); plt.xlabel('Department');
plt.ylabel('Count')
plt.tight_layout(); plt.show()

# — 3. Box Plot —————
plt.figure(figsize=(9, 5))
sns.boxplot(data=df, x='department', y='salary', palette='Set2')
plt.title('Salary Distribution by Department (Box Plot)')
plt.tight_layout(); plt.show()

# — 4. Seaborn KDE (Kernel Density Estimate) —————
plt.figure(figsize=(8, 4))
sns.kdeplot(df['age'], fill=True, color='steelblue', alpha=0.5, label='Age')
plt.title('Age Density Curve (KDE)'); plt.xlabel('Age')
plt.legend(); plt.tight_layout(); plt.show()

# — 5. Pair Plot —————
df_numeric = df[['age', 'salary']]
sns.pairplot(df, hue='department', diag_kind='kde', plot_kws={'alpha':0.5})
plt.suptitle('Pairplot: Age and Salary by Department', y=1.01)
plt.tight_layout(); plt.show()

```

Mini Quiz

Q1: What is the difference between a histogram and a bar chart?

 **Answer:** A histogram shows distribution of continuous numerical data (bars touch, representing ranges). A bar chart shows counts or values for discrete categories (bars separated, representing distinct groups).

Q2: A histogram of house prices is heavily right-skewed. What does this tell you?

✓ Answer: Most houses are clustered at lower prices, but a small number of very expensive houses pull the tail to the right. The mean will be higher than the median because of these expensive outliers.

Chapter 3: Measures of Central Tendency

3.1 The Three Measures

Central tendency measures describe the 'centre' or 'typical value' of a dataset with a single number. There are three main measures: mean, median, and mode.

The Mean — The Arithmetic Average

$$\text{Mean } (\mu) = (x_1 + x_2 + \dots + x_n) / n = (1/n) \sum x_i$$

Example: Data = [10, 20, 30, 40, 100]

$$\text{Mean} = (10+20+30+40+100)/5 = 200/5 = 40$$

Watch Out

The mean is pulled strongly by outliers. In the example above, removing 100 gives a mean of 25. A single extreme value changed the result by 60%! This is why the mean can be misleading for skewed data.

The Median — The Middle Value

Sort data, then:

Odd n: Median = middle value

Even n: Median = average of two middle values

Example (odd): [10, 20, 30, 40, 100] → Median = 30

Example (even): [10, 20, 30, 40] → Median = (20+30)/2 = 25

Key Insight

The median is resistant to outliers. In [10, 20, 30, 40, 100], the median is 30 whether that last value is 100 or 1,000,000. This is why median house prices and median salaries are reported instead of means — a few billionaires would inflate the mean enormously.

The Mode — The Most Frequent Value

Mode = the value that appears most often

Example: [2, 3, 3, 4, 4, 5] → Mode = 4
 Bimodal: [2, 2, 3, 5, 5] → Modes = 2 and 5
 No mode: [1, 2, 3, 4, 5] → All values unique

When to Use Which Measure

Mean: Symmetric data, no outliers — e.g. heights, test scores in a normal class

Median: Skewed data or outliers present — e.g. income, house prices, response times

Mode: Categorical data, or finding most common category — e.g. most popular product

ML context:

- Loss functions use the mean (MSE averages squared errors)
- Robust statistics use the median (Huber loss, median absolute deviation)
- Classification outputs use the mode (majority vote in random forests)

3.2 Python: Central Tendency

```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

# Dataset with outlier
data_normal = [10, 12, 14, 15, 15, 16, 17, 18, 20]
data_outlier = [10, 12, 14, 15, 15, 16, 17, 18, 200] # outlier added

for label, d in [('Normal data', data_normal), ('With outlier', data_outlier)]:
    arr = np.array(d, dtype=float)
    print(f'\n{label}:')
    print(f' Mean: {np.mean(arr):.2f}')
    print(f' Median: {np.median(arr):.2f}')
    print(f' Mode: {stats.mode(arr, keepdims=True).mode[0]:.2f}')

# Visualise the effect of outlier
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, d, title in zip(axes,
                        [data_normal, data_outlier],
                        ['Normal Data', 'Data With Outlier']):
    arr = np.array(d, dtype=float)
    ax.hist(arr, bins=12, color='steelblue', edgecolor='white', alpha=0.8)
    ax.axvline(np.mean(arr), color='red', lw=2,
    label=f'Mean={np.mean(arr):.1f}')
    ax.axvline(np.median(arr), color='green', lw=2,
    label=f'Median={np.median(arr):.1f}')
    ax.set_title(title); ax.legend(); ax.grid(axis='y', alpha=0.3)
plt.tight_layout(); plt.show()

# Real-world: California housing dataset
```

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing(as_frame=True)
prices = housing.data['AveRooms']
print(f'\nCA Housing - AveRooms:')
print(f'  Mean:    {prices.mean():.2f}')
print(f'  Median:  {prices.median():.2f}')
print(f'  Difference signals skew: {(prices.mean()-prices.median()):.2f}')
```

Mini Quiz

Q1: Dataset: [5, 7, 7, 8, 9, 100]. Which best describes the 'typical' value — mean or median?

 *Answer: Median = 7.5. Mean = 22.7 (dragged up by 100). Median is far better here — it represents where most of the data actually sits.*

Q2: In a neural network, the loss function MSE uses the mean. What risk does this create?

 *Answer: A single extreme prediction error (outlier) can dominate the MSE and cause the model to focus disproportionately on that one sample. Huber loss or MAE can be more robust alternatives.*

Chapter 4: Measures of Dispersion

4.1 Why Spread Matters

Two datasets can have the same mean but completely different spreads. Imagine two job offers: both pay an average of £50,000, but one ranges from £48,000 to £52,000 while the other ranges from £20,000 to £80,000. The spread matters enormously!

Range

$$\text{Range} = \text{Maximum} - \text{Minimum}$$

Simple but sensitive to outliers. A single extreme value can make the range misleading.

Variance — Average Squared Deviation

$$\text{Population Variance: } \sigma^2 = (1/N) \sum (x_i - \mu)^2$$

$$\text{Sample Variance: } s^2 = (1/(n-1)) \sum (x_i - \bar{x})^2 \quad \leftarrow \text{use } n-1 \text{ for unbiased estimate}$$

Example: Data = [2, 4, 4, 4, 5, 5, 7, 9] Mean = 5

Deviations: [-3, -1, -1, -1, 0, 0, 2, 4]

Squared: [9, 1, 1, 1, 0, 0, 4, 16]

$$\text{Variance: } (9+1+1+1+0+0+4+16)/8 = 32/8 = 4$$

Standard Deviation — The Most Used Measure

$$\sigma = \sqrt{\sigma^2} \quad \text{Population standard deviation}$$

$$s = \sqrt{s^2} \quad \text{Sample standard deviation}$$

Same units as original data (unlike variance which is squared)

$$\text{Example: Variance} = 4 \rightarrow \text{Standard deviation} = \sqrt{4} = 2$$

The 68-95-99.7 Rule (Empirical Rule)

For a normal distribution:

- 68% of data falls within 1 standard deviation of the mean ($\mu \pm \sigma$)
- 95% of data falls within 2 standard deviations of the mean ($\mu \pm 2\sigma$)
- 99.7% of data falls within 3 standard deviations of the mean ($\mu \pm 3\sigma$)

This rule is used in outlier detection: values beyond $\pm 3\sigma$ are potential outliers.
It also underpins Z-score normalisation and standardisation in ML.

Interquartile Range (IQR)

Q1 = 25th percentile (lower quartile)

Q3 = 75th percentile (upper quartile)

$IQR = Q3 - Q1$

Middle 50% of the data — resistant to outliers

Outlier rule: values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are flagged as outliers

Real-World Analogy

Standard deviation is like average deviation from the group norm. If the class average exam score is 70 with $\text{std}=5$, most students scored between 65 and 75. If $\text{std}=25$, scores are all over the place. A tight cluster = low std. A sprawling spread = high std.

4.2 Python: Dispersion & Box Plots

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

np.random.seed(7)
data = np.concatenate([np.random.normal(50, 10, 95), [95, 98, 3, 2, 1]]) # with outliers

# — Dispersion metrics —————
print('=== Dispersion Metrics ===')
print(f'Range:      {np.ptp(data):.2f}')
print(f'Variance:   {np.var(data, ddof=1):.2f}')
print(f'Std Dev:     {np.std(data, ddof=1):.2f}')
Q1, Q3 = np.percentile(data, [25, 75])
IQR = Q3 - Q1
print(f'Q1={Q1:.2f}  Q3={Q3:.2f}  IQR={IQR:.2f}')

# IQR-based outlier bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = data[(data < lower) | (data > upper)]
print(f'Outlier bounds: [{lower:.2f}, {upper:.2f}']')
```

```

print(f'Outliers found: {outliers}')

# — Box Plot —————
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].boxplot(data, patch_artist=True,
                boxprops=dict(facecolor='lightblue', color='navy'),
                medianprops=dict(color='red', linewidth=2))
axes[0].set_title('Box Plot with Outliers')
axes[0].set_ylabel('Value')

# Annotate quartiles
axes[0].text(1.1, np.median(data), f'Median={np.median(data):.1f}', color='red',
            va='center')
axes[0].text(1.1, Q1, f'Q1={Q1:.1f}', color='navy', va='center')
axes[0].text(1.1, Q3, f'Q3={Q3:.1f}', color='navy', va='center')

# — Histogram with std bands —————
mu, sigma = np.mean(data), np.std(data, ddof=1)
axes[1].hist(data, bins=20, density=True, color='steelblue', alpha=0.6,
            edgecolor='white')
for k, col, lbl in [(1, 'gold', '±1σ (68%)'), (2, 'orange', '±2σ (95%)'), (3, 'red', '±3σ (99.7%)')]:
    axes[1].axvspan(mu-k*sigma, mu+k*sigma, alpha=0.1, color=col, label=lbl)
axes[1].axvline(mu, color='navy', lw=2, label=f'Mean={mu:.1f}')
axes[1].set_title('Distribution with σ Bands'); axes[1].legend(fontsize=8)
plt.tight_layout(); plt.show()

```

Mini Quiz

Q1: Dataset A: mean=50, std=2. Dataset B: mean=50, std=25. Which has more spread?

✓ **Answer:** Dataset B has much more spread. std=25 means values are routinely 25 units from the mean. Dataset A is tightly clustered (± 2 around 50).

Q2: Why do we divide by (n-1) for sample variance instead of n?

✓ **Answer:** Using n slightly underestimates the true population variance (biased). Using $n-1$ corrects this bias (Bessel's correction), giving an unbiased estimate of the population variance from a sample.

Chapter 5: Shape of Data — Skewness and Kurtosis

5.1 Skewness — Asymmetry of Distribution

Skewness measures how asymmetric a distribution is around its mean. A perfectly symmetric distribution (like the normal curve) has skewness = 0.

Skewness	Shape	Mean vs Median	Real Example	ML Impact
Positive (right)	Long right tail	Mean > Median	Income, house prices	Log-transform often helps
Negative (left)	Long left tail	Mean < Median	Age at retirement	Reflect and log-transform
Zero	Symmetric	Mean ≈ Median	Heights, IQ scores	Most ML algorithms work well

$$\text{Skewness} = (1/n) \sum [(x_i - \bar{x})/s]^3 \quad (\text{third standardised moment})$$

Skewness > 0: right (positive) skew — tail extends right
Skewness < 0: left (negative) skew — tail extends left
Skewness = 0: symmetric

5.2 Kurtosis — Peakedness and Tail Weight

Kurtosis measures how peaked a distribution is and how heavy its tails are compared to a normal distribution (which has kurtosis = 3, or excess kurtosis = 0).

Kurtosis Type	Shape	Tail Weight	Example
Leptokurtic (>3)	Sharp peak, heavy tails	More extreme outliers	Stock market returns, neural net weights
Mesokurtic (=3)	Normal peak	Normal tails	Heights, IQ scores
Platykurtic (<3)	Flat peak, thin tails	Fewer extremes	Uniform distributions



Skewness and Kurtosis in ML

Feature engineering: Highly skewed features cause problems for linear models. Log-transform, sqrt, or Box-Cox transformation reduces skewness and improves model performance.

Neural networks: Weight initialisation should avoid very leptokurtic distributions — heavy tails cause exploding gradients. Xavier and He initialisation use carefully chosen variances.

Anomaly detection: High kurtosis signals heavy tails — extreme events are more common than a Gaussian model would predict. This matters for fraud detection and risk modelling.

5.3 Python: Skewness, Kurtosis, and Transformations

```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(0)

# Three distributions with different shapes
normal_data = np.random.normal(0, 1, 1000)
right_skewed = np.random.exponential(scale=2, size=1000) # right skew
left_skewed = -np.random.exponential(scale=2, size=1000) + 10 # left skew

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

datasets = [(normal_data, 'Normal', 'blue'),
            (right_skewed, 'Right-Skewed', 'red'),
            (left_skewed, 'Left-Skewed', 'green')]

for i, (d, name, col) in enumerate(datasets):
    sk = stats.skew(d)
    ku = stats.kurtosis(d) # excess kurtosis
    axes[0,i].hist(d, bins=40, color=col, alpha=0.7, edgecolor='white')
    axes[0,i].axvline(np.mean(d), color='black', lw=2, linestyle='--',
label='Mean')
    axes[0,i].axvline(np.median(d), color='orange',lw=2, linestyle='-',
label='Median')
    axes[0,i].set_title(f'{name}\nSkew={sk:.2f}, Kurt={ku:.2f}')
    axes[0,i].legend(fontsize=8)

# Log transformation on right-skewed data
log_transformed = np.log1p(right_skewed) # log1p = log(1+x) handles 0s

axes[1,0].hist(right_skewed, bins=40, color='red', alpha=0.7,
edgecolor='white')
axes[1,0].set_title(f'Before Log Transform\nSkew={stats.skew(right_skewed):.2f}')
axes[1,1].hist(log_transformed, bins=40, color='purple', alpha=0.7,
edgecolor='white')
axes[1,1].set_title(f'After Log
Transform\nSkew={stats.skew(log_transformed):.2f}')

# Q-Q plot: check if data is normal
stats.probplot(log_transformed, dist='norm', plot=axes[1,2])
axes[1,2].set_title('Q-Q Plot: Log-Transformed Data')

plt.tight_layout(); plt.show()

# Summary table
```

```
for d, name in [(normal_data, 'Normal'), (right_skewed, 'Right-Skewed'),
               (left_skewed, 'Left-Skewed')]:
    print(f'{name:15s}: skew={stats.skew(d):+.3f}
          kurtosis={stats.kurtosis(d):+.3f}')
```

Mini Quiz

Q1: House prices are right-skewed. Should you use mean or median to report the 'typical' price?

✓ *Answer: Median. Right-skew means a few very expensive homes pull the mean up. The median (middle value) represents what a 'typical' home costs far more accurately.*

Q2: Why does log-transforming a right-skewed feature often improve a linear regression model?

✓ *Answer: Linear regression assumes errors are normally distributed. Highly skewed features violate this, creating heteroscedasticity. Log-transform makes the distribution more symmetric, meeting model assumptions better.*

Chapter 6: The Normal Distribution

6.1 What is the Normal Distribution?

The normal (Gaussian) distribution is the most important distribution in statistics and machine learning. It describes countless natural phenomena and underlies most classical statistical methods.

$$f(x) = (1 / \sqrt{2\pi\sigma^2}) \cdot \exp(-(x-\mu)^2 / (2\sigma^2))$$

Parameters:

μ (mu) = mean — controls the centre (peak)

σ (sigma) = standard deviation — controls the width

σ^2 = variance

Notation: $X \sim N(\mu, \sigma^2)$

Real-World Analogy

The normal distribution appears everywhere because of the Central Limit Theorem: the average of many independent random variables tends toward a normal distribution, regardless of what distribution those variables have. Average test scores, average heights, measurement errors, random walks — all become approximately normal when averaged over enough observations.

Properties of the Normal Distribution

- Perfectly symmetric around the mean (skewness = 0)
- Mean = Median = Mode (all at the peak)
- Defined by just two parameters: μ and σ
- 68-95-99.7 rule holds
- The Standard Normal $N(0,1)$ has $\mu=0$, $\sigma=1$

6.2 Why ML Assumes Normality

Algorithm/Concept	How Normality Is Assumed
Linear Regression	Errors (residuals) assumed normally distributed
Neural Net Weight Init	He/Xavier init: weights $\sim N(0, \sigma^2)$ for stable training
Batch Normalization	Normalises layer inputs toward $N(0,1)$ to speed training

Naive Bayes (Gaussian)	Feature likelihoods modelled as Gaussian distributions
Gradient Descent	SGD noise approximated as Gaussian — helps escaping saddle points
Feature Scaling	Z-score standardisation transforms features toward $N(0,1)$

6.3 The Z-Score — Standardising Values

$$z = (x - \mu) / \sigma$$

Meaning: how many standard deviations is x from the mean?

$z = 0$: value equals the mean

$z = 2$: value is 2 standard deviations above the mean

$z = -1$: value is 1 standard deviation below the mean

6.4 Python: Normal Distribution

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

x = np.linspace(-4, 4, 400)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# — 1. Effect of changing  $\mu$  —————
for mu, col in [(-2, 'blue'), (0, 'red'), (2, 'green')]:
    y = stats.norm.pdf(x, mu, 1)
    axes[0].plot(x, y, color=col, lw=2, label=f' $\mu={mu}$ ,  $\sigma=1$ ')
axes[0].set_title('Effect of Mean ( $\mu$ )'); axes[0].legend(); axes[0].grid(alpha=0.3)

# — 2. Effect of changing  $\sigma$  —————
for sigma, col in [(0.5, 'blue'), (1, 'red'), (2, 'green')]:
    y = stats.norm.pdf(x, 0, sigma)
    axes[1].plot(x, y, color=col, lw=2, label=f' $\mu=0$ ,  $\sigma={sigma}$ ')
axes[1].set_title('Effect of Std Dev ( $\sigma$ )'); axes[1].legend();
axes[1].grid(alpha=0.3)

# — 3. 68-95-99.7 Rule visualisation —————
y = stats.norm.pdf(x, 0, 1)
axes[2].plot(x, y, 'k-', lw=2)
for k, col, label in [(1, 'lightblue', '68%'), (2, 'steelblue', '95%'), (3, 'navy', '99.7%')]:
    mask = (x >= -k) & (x <= k)
    axes[2].fill_between(x[mask], y[mask], alpha=0.4, color=col, label=f' $\pm{k}\sigma$ : {label}')
axes[2].set_title('68-95-99.7 Rule'); axes[2].legend(fontsize=8);
axes[2].grid(alpha=0.3)
```

```
plt.suptitle('Normal Distribution Properties', fontsize=14, y=1.02)
plt.tight_layout(); plt.show()

# Z-score calculation example
heights = np.array([160, 165, 170, 175, 180, 185, 190])
mu, sigma = heights.mean(), heights.std(ddof=1)
z_scores = (heights - mu) / sigma
print('Heights: ', heights)
print(f'Mean={mu:.1f}, Std={sigma:.2f}')
print('Z-scores: ', np.round(z_scores, 2))
```

Mini Quiz

Q1: A student scores 85 on a test where $\mu=70$ and $\sigma=10$. What is their Z-score and what does it mean?

✓ *Answer: $Z = (85-70)/10 = 1.5$. They scored 1.5 standard deviations above the mean — better than about 93% of students.*

Q2: Why does Batch Normalisation in deep learning re-centre activations toward $N(0,1)$?

✓ *Answer: Activations that drift to large magnitudes cause vanishing/exploding gradients. Normalising to $N(0,1)$ keeps gradient magnitudes healthy, allows higher learning rates, and makes training much faster and more stable.*

Chapter 7: Outliers and Data Scaling

7.1 What are Outliers?

Outliers are data points that are significantly different from the rest of the dataset. They can be genuine rare events, measurement errors, or data entry mistakes. Handling them correctly is crucial for model quality.

Method 1: Z-Score Method

Flag as outlier if: $|z| = |x - \mu| / \sigma > 3$

Assumes approximately normal distribution.

Points beyond 3 standard deviations are extreme (only 0.3% in a true normal).

Method 2: IQR Method (More Robust)

Lower fence = $Q1 - 1.5 \times IQR$

Upper fence = $Q3 + 1.5 \times IQR$

Values outside these fences are flagged as outliers.

Does NOT assume normality — works for skewed distributions.

Strategy	When to Use	Risk
Remove outlier	Confirmed data error or irrelevant	Lose valid data points
Cap/winsorize	Extreme values but possibly real	Distorts extreme cases
Log-transform	Right-skewed data with natural outliers	Must handle zeros separately
Use robust algorithms	Many genuine outliers expected	May reduce predictive power
Keep and model	Outliers ARE the signal (fraud detection)	Model must handle extremes

7.2 Data Scaling and Normalisation

Most ML algorithms are sensitive to the scale of features. When one feature ranges 0–1 and another ranges 0–100,000, the model may unfairly weight the larger-scale feature. Scaling fixes this.

Min-Max Normalisation (0 to 1 scaling)

$$x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Result: all values in [0, 1]

Use when: you need bounded output, features have non-Gaussian distributions

Risk: sensitive to outliers (a single extreme value compresses all others)

Standardisation (Z-score scaling)

$$x_{\text{scaled}} = (x - \mu) / \sigma$$

Result: mean = 0, std = 1 (but NOT bounded)

Use when: algorithm assumes Gaussian input (SVM, logistic reg, neural nets)

Advantage: less affected by outliers than min-max

Why Scaling Affects Gradient Descent

Imagine a loss surface shaped like an elongated valley. One feature ranges 0-1, another 0-1000.

The gradient descent steps will bounce erratically between features because the loss landscape is extremely elongated.

After scaling both to the same range:

- Loss surface becomes more 'circular'
- Gradient steps are balanced and efficient
- Learning rate can be set higher
- Convergence is much faster (often 10-100x)

Tree-based models (Random Forest, XGBoost) are scale-invariant — no scaling needed.

Distance-based models (k-NN, k-Means, SVM) are highly sensitive to scale.

7.3 Python: Outlier Detection and Scaling

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
data = np.concatenate([np.random.normal(50, 8, 200), [95, 98, 5, 3]]) # with outliers
```

```

# — Outlier Detection —————
# Z-score method
z_scores = (data - data.mean()) / data.std(ddof=1)
z_outliers = data[np.abs(z_scores) > 3]
print('Z-score outliers:', z_outliers)

# IQR method
Q1, Q3 = np.percentile(data, [25, 75])
IQR = Q3 - Q1
iqr_outliers = data[(data < Q1-1.5*IQR) | (data > Q3+1.5*IQR)]
print('IQR outliers:      ', iqr_outliers)

# — Scaling Comparison —————
feature = data.reshape(-1, 1)

mm_scaler = MinMaxScaler()
std_scaler = StandardScaler()

mm_scaled = mm_scaler.fit_transform(feature).flatten()
std_scaled = std_scaler.fit_transform(feature).flatten()

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

for ax, d, title, col in zip(axes,
                             [data, mm_scaled, std_scaled],
                             ['Original Data', 'Min-Max Scaled [0,1]', 'Standardised (z-score)'],
                             ['steelblue', 'green', 'coral']):
    ax.hist(d, bins=25, color=col, alpha=0.8, edgecolor='white')
    ax.set_title(f'{title}\nMean={np.mean(d):.2f}, Std={np.std(d):.2f}')
    ax.grid(axis='y', alpha=0.3)

plt.tight_layout(); plt.show()

# — Sklearn Pipeline example —————
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression()),
])
print('\nPipeline created successfully:', pipe.steps)

```

Mini Quiz

Q1: Should you apply min-max scaling or standardisation when a feature has extreme outliers?

 **Answer:** Standardisation is safer. Min-max scaling maps the full range [min, max] to [0,1] — one outlier at 1,000,000 would compress all other values near zero. Standardisation uses mean and std, which are more stable.

Q2: You're using k-Nearest Neighbours. Feature A ranges 0–1 and Feature B ranges 0–1000. Without scaling, which feature dominates distance calculations and why?

✓ Answer: Feature B dominates. Euclidean distance is $\sqrt{(\Delta A^2 + \Delta B^2)}$. Since ΔB can be up to 1000 vs ΔA up to 1, Feature B's contribution dwarfs Feature A. The algorithm effectively ignores Feature A entirely.

Chapter 8: Correlation and Covariance

8.1 Covariance — Direction of Relationship

$$\text{Cov}(X,Y) = (1/(n-1)) \sum (x_i - \bar{x})(y_i - \bar{y})$$

$\text{Cov} > 0$: when X increases, Y tends to increase (positive relationship)

$\text{Cov} < 0$: when X increases, Y tends to decrease (negative relationship)

$\text{Cov} = 0$: no linear relationship

Problem: units are not interpretable (e.g., 'kg·cm') — use correlation instead.

8.2 Pearson Correlation — Standardised Covariance

$$r = \text{Cov}(X,Y) / (\sigma_x \times \sigma_y)$$

Always between -1 and +1:

$r = +1$: perfect positive linear relationship

$r = -1$: perfect negative linear relationship

$r = 0$: no linear relationship

Rule of thumb: $|r| < 0.3 \rightarrow$ weak | $0.3-0.7 \rightarrow$ moderate | $> 0.7 \rightarrow$ strong

Correlation Value	Strength	Example Pair
0.9 to 1.0	Very Strong Positive	Height and weight
0.7 to 0.9	Strong Positive	Study hours and exam score
0.3 to 0.7	Moderate Positive	Income and happiness
0.0 to 0.3	Weak / None	Shoe size and IQ
-0.3 to 0.0	Weak Negative	Stress and sleep quality
-0.7 to -0.3	Moderate Negative	Exercise and body fat %
-1.0 to -0.7	Strong Negative	Speed and travel time

8.3 Correlation vs Causation

 **Watch Out**

Correlation $\neq$ Causation! This is one of the most important principles in data science. Two variables can be correlated without one causing the other. Always look for confounding variables and domain knowledge before claiming causation.

Famous Spurious Correlations

- Ice cream sales correlate with drowning deaths (both rise in summer — confound: temperature)
- Nicolas Cage movies correlate with pool drownings (pure coincidence — thousands of correlations exist)
- Shoe size correlates with reading ability in children (both increase with age — confound: age)

In ML: a feature might correlate with the target but only because both share a hidden cause. Including it in a model can hurt interpretability even if it improves raw accuracy.

8.4 Python: Correlation Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing

# Load California housing dataset
housing = fetch_california_housing(as_frame=True)
df = housing.frame
df.columns = [c.replace(' ', '_') for c in df.columns]

# — 1. Correlation Matrix —————
corr_matrix = df.corr(numeric_only=True)
print('Correlations with house price (MedHouseVal):')
print(corr_matrix['MedHouseVal'].sort_values(ascending=False))

# — 2. Correlation Heatmap —————
plt.figure(figsize=(10, 8))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool)) # upper triangle
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt='.2f',
            cmap='RdYlBu', center=0, square=True,
            linewidths=0.5, cbar_kws={'shrink':0.8})
plt.title('Feature Correlation Matrix - California Housing', fontsize=13)
plt.tight_layout(); plt.show()

# — 3. Scatter Plot Matrix (key features) —————
key_features = ['MedInc', 'AveRooms', 'HouseAge', 'MedHouseVal']
sns.pairplot(df[key_features], plot_kws={'alpha':0.2, 's':5},
            diag_kind='kde', corner=True)
plt.suptitle('Pairwise Scatter Plots - Key Features', y=1.01)
plt.tight_layout(); plt.show()
```

```
# — 4. Single Scatter + Regression Line —————
plt.figure(figsize=(7, 5))
plt.scatter(df['MedInc'], df['MedHouseVal'], alpha=0.1, s=5, color='steelblue')

# Add regression line
from numpy.polynomial.polynomial import polyfit
x_line = np.linspace(df['MedInc'].min(), df['MedInc'].max(), 100)
b, m = polyfit(df['MedInc'], df['MedHouseVal'], 1)
plt.plot(x_line, b + m*x_line, 'r-', lw=2, label='Regression line')
r = df['MedInc'].corr(df['MedHouseVal'])
plt.title(f'Median Income vs House Price (r = {r:.3f})')
plt.xlabel('Median Income'); plt.ylabel('House Price')
plt.legend(); plt.tight_layout(); plt.show()
```

Mini Quiz

Q1: Two features X and Y have a correlation of 0.95. Should you include both in a linear regression?

✓ *Answer: Probably not — this is multicollinearity. Both features carry nearly the same information, making coefficient estimates unstable. Drop one, use PCA to combine them, or use Ridge/Lasso regularisation.*

Q2: Sales of sunscreen and ice cream are positively correlated. Does sunscreen cause ice cream sales?

✓ *Answer: No — both are caused by a confounding variable: hot weather. When it's hot, both increase simultaneously. This is spurious correlation, not causation.*

Chapter 9: Sampling and Hypothesis Testing

9.1 Population vs Sample

In statistics, the population is the entire group you care about (all customers, all patients, all transactions). A sample is the subset you actually measure. Since we rarely have access to complete populations, we make inferences from samples.

Concept	Symbol	Meaning
Population mean	μ (mu)	True average for entire population
Sample mean	$\bar{x}$ (x-bar)	Average computed from your sample — estimates μ
Population std	σ (sigma)	True spread of entire population
Sample std	s	Spread computed from sample — estimates σ
Population size	N	Total elements in population
Sample size	n	Elements in your sample

Sampling Bias

Watch Out

Sampling bias occurs when the sample is not representative of the population. A famous example: early 20th century polls showed wealthy people overwhelmingly preferred Republican candidates — because they only surveyed people with phones and cars, who were wealthy! ML datasets suffer from similar bias — if training data over-represents certain groups, models will too.

9.2 Hypothesis Testing — The Basic Idea

Hypothesis testing is a framework for making decisions from data. We start with a claim (the null hypothesis), collect data, and decide whether the data gives enough evidence to reject that claim.

The Two Hypotheses

H_0 (Null Hypothesis): The 'boring' claim — nothing interesting is happening.

Example: 'This new drug has no effect on blood pressure.'

H_1 (Alternative Hypothesis): The interesting claim we want to detect.

Example: 'This new drug DOES reduce blood pressure.'

We start by ASSUMING H_0 is true, then test whether the data is consistent with it.
If the data is very unlikely under H_0 , we reject H_0 in favour of H_1 .

The p-value — Plain English

The p-value is the probability of observing results as extreme as yours if the null hypothesis were actually true. A very small p-value means your results would be extremely unlikely if H_0 were true — which gives evidence against H_0 .

p-value Interpretation

$p < 0.05$: 'Statistically significant' — strong evidence against H_0 . Reject H_0 .

$p \geq 0.05$: 'Not significant' — insufficient evidence to reject H_0 .

⚠ p-value does NOT tell you the probability H_0 is true.

⚠ Statistical significance $\neq$ practical significance.

⚠ $p < 0.05$ is a convention, not a law — always consider effect size too.

9.3 Python: Hypothesis Testing

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

np.random.seed(42)

# Scenario: Does a new training algorithm improve test accuracy?
# Old algorithm: 50 trials
# New algorithm: 50 trials
old_algo_scores = np.random.normal(82, 5, 50) #  $\mu=82$ ,  $\sigma=5$ 
new_algo_scores = np.random.normal(85, 5, 50) #  $\mu=85$ ,  $\sigma=5$ 

print('Old algorithm: mean={:.2f}, std={:.2f}'.format(old_algo_scores.mean(),
old_algo_scores.std()))
print('New algorithm: mean={:.2f}, std={:.2f}'.format(new_algo_scores.mean(),
new_algo_scores.std()))

# Two-sample t-test
t_stat, p_value = stats.ttest_ind(old_algo_scores, new_algo_scores)
print(f'\nt-statistic: {t_stat:.4f}')
print(f'p-value: {p_value:.6f}')

if p_value < 0.05:
    print('Result: REJECT  $H_0$  — new algorithm is significantly better ( $p<0.05$ )')
```

```

else:
    print('Result: FAIL to reject  $H_0$  – insufficient evidence of improvement')

# Effect size (Cohen's d)
pooled_std = np.sqrt((old_algo_scores.std()**2 + new_algo_scores.std()**2) / 2)
cohens_d = (new_algo_scores.mean() - old_algo_scores.mean()) / pooled_std
print(f"Cohen's d: {cohens_d:.3f} (>0.8 = large effect)")

# A/B Testing: Chi-square test for proportions
# Example: two ad variants, did click-through rates differ?
# Version A: 1000 shown, 120 clicked (12%)
# Version B: 1000 shown, 145 clicked (14.5%)
observed = np.array([[120, 880], [145, 855]]) # [[clicks, no-clicks], ...]
chi2, p, dof, expected = stats.chi2_contingency(observed)
print(f'\nA/B Test – Chi-square: {chi2:.4f}, p-value: {p:.4f}')
if p < 0.05: print('Versions differ significantly!')
else:        print('No significant difference between versions.')

```

Mini Quiz

Q1: An ML experiment has p-value = 0.03. What does this mean?

✓ *Answer: If the null hypothesis (no improvement) were true, we'd see results this extreme only 3% of the time by chance. Since $3\% < 5\%$, we have significant evidence to reject H_0 — the improvement is statistically real.*

Q2: Why is sample size important in hypothesis testing and ML?

✓ *Answer: Small samples: high variance, unreliable estimates, hard to detect real effects. Large samples: can detect even tiny effects that may be practically meaningless. Always report both p-value AND effect size.*

Chapter 10: Linear Regression

10.1 The Line of Best Fit

Linear regression finds the straight line through data that minimises the prediction errors. It assumes a linear relationship between input features (X) and the output (y).

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_px_p$$

$\hat{y}$ = predicted value

w_0 = intercept (value of $\hat{y}$ when all $x_i=0$)

$w_1, \dots, w_p$ = slopes (how much $\hat{y}$ changes per unit increase in each x_i)

10.2 Residuals — The Errors

$$\text{Residual} = \text{Actual} - \text{Predicted} = y_i - \hat{y}_i$$

Least Squares: minimise $\sum (y_i - \hat{y}_i)^2$ — sum of squared residuals

Optimal weights: $w^* = (X^T X)^{-1} X^T y$ (Normal Equation)

10.3 Model Evaluation Metrics

Metric	Formula	Meaning	Range	Better When
MAE	$(1/n) \sum y_i - \hat{y}_i $	Mean absolute error — same units	$0 \rightarrow \infty$	Lower
MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	Mean squared error — penalises big errors	$0 \rightarrow \infty$	Lower
RMSE	VMSE	Root MSE — same units as target	$0 \rightarrow \infty$	Lower
R^2	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	% variance explained by model	$0 \rightarrow 1$	Higher (closer to 1)

Understanding R^2 (R-squared)

$R^2 = 0$: Model explains nothing — as useless as predicting the mean every time

$R^2 = 0.5$: Model explains 50% of variance in the target

$R^2 = 1.0$: Model explains 100% — perfect fit (rare in real data)

R^2 can be NEGATIVE if the model is worse than a horizontal line (constant prediction).

A high R^2 on training data with low R^2 on test data = OVERFITTING.

10.4 Python: Complete Regression

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing

# Load data
housing = fetch_california_housing(as_frame=True)
X = housing.data[['MedInc', 'AveRooms', 'HouseAge', 'AveOccup']]
y = housing.target # House price

# Train / test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Scale features
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Fit model
model = LinearRegression()
model.fit(X_train_s, y_train)

# Predict
y_pred = model.predict(X_test_s)

# — Metrics —————
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print('=== Regression Metrics ===')
print(f'MAE: {mae:.4f} (average error in $100k units)')
print(f'MSE: {mse:.4f}')
print(f'RMSE: {rmse:.4f}')
print(f'R²: {r2:.4f} ({r2*100:.1f}% variance explained)')

print('\nFeature coefficients:')
for feat, coef in zip(X.columns, model.coef_):
    print(f' {feat:12s}: {coef:+.4f}')

# — Residual Plot —————
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].scatter(y_test, y_pred, alpha=0.2, s=5, color='steelblue')
```

```
axes[0].plot([y_test.min(), y_test.max()],
             [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect fit')
axes[0].set_xlabel('Actual Price'); axes[0].set_ylabel('Predicted Price')
axes[0].set_title(f'Actual vs Predicted (R²={r2:.3f})')
axes[0].legend()

residuals = y_test - y_pred
axes[1].scatter(y_pred, residuals, alpha=0.2, s=5, color='coral')
axes[1].axhline(0, color='black', lw=2, linestyle='--')
axes[1].set_xlabel('Predicted'); axes[1].set_ylabel('Residual')
axes[1].set_title('Residual Plot (should be random scatter)')

plt.tight_layout(); plt.show()
```

Mini Quiz

Q1: Model A: RMSE = 2.5. Model B: RMSE = 4.1 (same dataset). Which is better?

✓ *Answer: Model A. Lower RMSE means smaller average prediction errors. RMSE of 2.5 means predictions are off by about 2.5 units on average.*

Q2: Your regression model has $R^2=0.97$ on training data and $R^2=0.42$ on test data. What is happening?

✓ *Answer: Severe overfitting. The model has memorised training data patterns including noise, but fails to generalise to new data. Solutions: regularisation (Ridge/Lasso), more data, simpler model, cross-validation.*

Chapter 11: Exploratory Data Analysis (EDA)

11.1 What is EDA and Why?

Exploratory Data Analysis (EDA) is the process of examining and summarising a dataset before building any models. It helps you understand the data's structure, find patterns, spot problems, and guide modelling decisions. EDA was popularised by statistician John Tukey.

The 8 Goals of EDA

1. Understand the shape and size of the dataset (rows, columns, types)
2. Find missing values and decide how to handle them
3. Detect outliers and decide whether to remove/transform them
4. Understand distributions of each feature (histograms, box plots)
5. Discover relationships between features (correlation, scatter plots)
6. Understand the target variable (class balance, distribution)
7. Generate hypotheses for feature engineering
8. Validate assumptions required by ML models

11.2 The EDA Toolkit

EDA Tool	What It Shows	Python Method
df.shape	Number of rows and columns	Quick sanity check
df.dtypes	Data type of each column	Identifies encoding needs
df.describe()	Count, mean, std, min, quartiles, max	Fast stats overview
df.isnull().sum()	Missing value counts per column	First cleaning step
df.nunique()	Number of unique values per column	Spot high-cardinality cols
df.value_counts()	Frequency of each category	Check class balance
Histogram	Distribution of a numeric variable	plt.hist / sns.histplot
Box plot	Spread + outliers by group	sns.boxplot
Heatmap	Correlation between all features	sns.heatmap(df.corr())
Scatter plot	Relationship between two variables	plt.scatter

11.3 Python: Full EDA Workflow

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer

# — Load Data —————
cancer = load_breast_cancer(as_frame=True)
df = cancer.frame
df['target_name'] = df['target'].map({0:'Malignant', 1:'Benign'})

print('Dataset shape:', df.shape)
print('Target distribution:')
print(df['target_name'].value_counts())

# — 1. Overview —————
print('\n=== Basic Info ===')
print(df.describe().round(2).to_string())
print('\nMissing values:')
print(df.isnull().sum()[df.isnull().sum() > 0])

# — 2. Class Balance —————
fig, axes = plt.subplots(2, 3, figsize=(15, 9))

df['target_name'].value_counts().plot(kind='bar', ax=axes[0,0],
color=['coral','steelblue'], edgecolor='white')
axes[0,0].set_title('Class Distribution');
axes[0,0].set_xticklabels(axes[0,0].get_xticklabels(), rotation=0)

# — 3. Feature Distributions —————
key_features = ['mean radius', 'mean texture', 'mean area', 'mean smoothness']
for ax, feat in zip(axes.flat[1:5], key_features):
    for label, grp in df.groupby('target_name'):
        ax.hist(grp[feat], bins=25, alpha=0.6, label=label, edgecolor='white')
        ax.set_title(feat); ax.legend(fontsize=8); ax.grid(axis='y', alpha=0.3)

# — 4. Correlation Heatmap (top features) —————
top_feats = list(cancer.feature_names[:10]) + ['target']
corr = df[top_feats].corr(numeric_only=True)
plt.figure(figsize=(11, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlBu', center=0,
            linewidths=0.5, square=True, cbar_kws={'shrink':0.7})
plt.title('Correlation Matrix - Top 10 Features + Target', fontsize=13)
plt.tight_layout(); plt.show()

# — 5. Box Plots by Class —————
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
for ax, feat in zip(axes, key_features):
    sns.boxplot(data=df, x='target_name', y=feat, ax=ax,
                palette=['coral','steelblue'])
    ax.set_title(feat); ax.set_xlabel('')
plt.suptitle('Feature Distribution by Diagnosis', y=1.01, fontsize=13)
plt.tight_layout(); plt.show()

# — 6. Statistical Summary per Class —————
print('\n=== Mean values by class ===')
print(df.groupby('target_name')[key_features].mean().round(3))
```

Mini Quiz

Q1: During EDA you find 15% of values in an important feature are missing. What are your options?

✓ Answer: (1) Drop the rows if missing is random and dataset is large. (2) Impute with mean/median/mode (simple). (3) Use KNN or model-based imputation (better). (4) Create a 'missing' indicator feature. The right choice depends on WHY values are missing.

Q2: A binary classification dataset has 95% of samples in class 0 and 5% in class 1. Why is this problematic and how can you address it?

✓ Answer: Class imbalance: a model can get 95% accuracy by predicting class 0 always! Solutions: oversample minority (SMOTE), undersample majority, use class weights in the loss function, use AUC-ROC instead of accuracy.

Chapter 12: Bias-Variance Tradeoff

12.1 The Central Challenge of Machine Learning

The bias-variance tradeoff is the most important concept in understanding why ML models fail and how to fix them. Every model has two sources of error: bias and variance. You cannot minimise both simultaneously — reducing one tends to increase the other.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Bias: Error from wrong assumptions in the model

Variance: Error from sensitivity to small fluctuations in training data

Noise: Irreducible error — inherent randomness in the data (can't fix)

Term	Cause	Symptom	Fix
High Bias (Underfitting)	Model too simple	Poor training AND test accuracy	More complex model, more features, reduce regularisation
High Variance (Overfitting)	Model too complex	Good training, poor test accuracy	Regularisation, dropout, more data, simpler model
Good Fit	Balanced	Good training and test (similar) accuracy	Continue improving generalisation

Real-World Analogy

Bias-Variance is like archery. High bias = arrows consistently hit far LEFT of the target (systematically wrong). High variance = arrows are scattered all over the target (inconsistent). Good model = arrows tightly clustered around the bullseye (accurate and consistent). The goal is low bias AND low variance, but they fight each other.

12.2 Visualising the Tradeoff

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, learning_curve
from sklearn.metrics import mean_squared_error

np.random.seed(42)
```

```

# True function: y = sin(x) + noise
n = 120
X = np.sort(np.random.uniform(0, 2*np.pi, n))
y = np.sin(X) + np.random.normal(0, 0.3, n)

X_train, X_test, y_train, y_test = train_test_split(X.reshape(-1,1), y,
test_size=0.3)

degrees = [1, 3, 10, 20]
x_plot = np.linspace(0, 2*np.pi, 300).reshape(-1,1)

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, deg in zip(axes, degrees):
    pipe = Pipeline(['poly', PolynomialFeatures(deg)], ('lr',
LinearRegression()))
    pipe.fit(X_train, y_train)
    y_hat_tr = pipe.predict(X_train)
    y_hat_te = pipe.predict(X_test)
    tr_mse = mean_squared_error(y_train, y_hat_tr)
    te_mse = mean_squared_error(y_test, y_hat_te)

    ax.scatter(X_train, y_train, s=10, alpha=0.4, label='Train')
    ax.scatter(X_test, y_test, s=10, alpha=0.4, color='coral', label='Test')
    ax.plot(x_plot, pipe.predict(x_plot), 'navy', lw=2)
    ax.set_title(f'Degree {deg}\nTrain MSE={tr_mse:.2f} Test MSE={te_mse:.2f}',
fontsize=9)
    ax.set_ylim(-2.5, 2.5); ax.legend(fontsize=7)

axes[0].set_title(axes[0].get_title() + '\n← UNDERFITTING', fontsize=9)
axes[-1].set_title(axes[-1].get_title() + '\n← OVERFITTING', fontsize=9)
plt.suptitle('Bias-Variance: Polynomial Degree Effect', fontsize=13, y=1.01)
plt.tight_layout(); plt.show()

# Learning Curves: diagnose bias vs variance
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, deg, title in [(axes[0], 1, 'Degree 1 (High Bias)'),
(axes[1], 8, 'Degree 8 (High Variance)')]:
    pipe = Pipeline(['poly', PolynomialFeatures(deg)], ('lr',
LinearRegression()))
    sizes, tr_scores, te_scores = learning_curve(
        pipe, X.reshape(-1,1), y,
        train_sizes=np.linspace(0.1,1.0,10), cv=5,
        scoring='neg_mean_squared_error')
    ax.plot(sizes, -tr_scores.mean(1), label='Train MSE', color='blue', lw=2)
    ax.plot(sizes, -te_scores.mean(1), label='Val MSE', color='red', lw=2)
    ax.fill_between(sizes, -tr_scores.mean(1)-tr_scores.std(1),
                    -tr_scores.mean(1)+tr_scores.std(1), alpha=0.15,
color='blue')
    ax.set_title(title); ax.set_xlabel('Training Size'); ax.set_ylabel('MSE')
    ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()

```

Mini Quiz

Q1: Training loss is 0.01 but validation loss is 1.8. What type of error is this and what should you do?

✓ Answer: High variance / overfitting. The model memorised training data but can't generalise. Solutions: regularisation (L1/L2), dropout, early stopping, data augmentation, cross-validation, simpler architecture.

Q2: Training loss is 0.7 and validation loss is also 0.72. Both are high. What is this called and how do you fix it?

✓ Answer: High bias / underfitting. The model is too simple to capture the pattern. Solutions: more complex model, additional features, feature engineering, reduce regularisation, train longer.

Chapter 13: Statistics Powering ML Models

13.1 Linear Regression — A Statistical Foundation

From a statistical perspective, linear regression is a probabilistic model: it assumes $y = w^T x + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$. The normal equations don't just minimise error — they compute the maximum likelihood estimate of the weights.

13.2 Naive Bayes — Pure Probability

Naive Bayes classifiers are among the purest statistical ML models. They apply Bayes' theorem with the 'naive' assumption that features are conditionally independent given the class.

$$P(\text{class} \mid \text{features}) \propto P(\text{class}) \times \prod P(\text{feature}_i \mid \text{class})$$

$P(\text{class})$: Prior — how common is each class?

$P(\text{feature}_i \mid \text{class})$: Likelihood — how likely is this feature given the class?

$P(\text{class} \mid \text{features})$: Posterior — updated belief after seeing the features

13.3 k-Means Clustering — Statistics in Unsupervised Learning

k-Means minimises the within-cluster sum of squared distances (variance). Each cluster centroid is the mean of all points in that cluster — a fundamental statistical quantity.

$$\text{Objective: minimise } \sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Each step: assign each point to nearest centroid (min Euclidean distance)

Update: $\mu_k \leftarrow$ mean of all points assigned to cluster k

13.4 Statistics in Neural Networks

NN Component	Statistical Concept	Why It Matters
Batch Normalisation	Z-score standardisation per layer	Prevents gradient issues, stabilises training
Dropout	Bayesian model averaging	Reduces overfitting, acts as regularisation
Weight Initialisation	Normal/uniform distribution sampling	Prevents vanishing/exploding gradients
Loss Functions	Negative log-likelihood	Cross-entropy = MLE for classification

L2 Regularisation	Gaussian prior on weights (MAP)	Controls model complexity, reduces variance
Data Augmentation	Bootstrap sampling	Increases effective training set size

13.5 Python: Statistics for ML Evaluation

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

iris = load_iris(as_frame=True)
X, y = iris.data, iris.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# — 1. Naive Bayes —————
gnb = GaussianNB()
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
nb_scores = cross_val_score(gnb, X_scaled, y, cv=cv, scoring='accuracy')
print('Naive Bayes CV Accuracy: {:.3f} ± {:.3f}'.format(nb_scores.mean(),
nb_scores.std()))

# — 2. Random Forest —————
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_scores = cross_val_score(rf, X_scaled, y, cv=cv, scoring='accuracy')
print('Random Forest CV Accuracy: {:.3f} ± {:.3f}'.format(rf_scores.mean(),
rf_scores.std()))

# — 3. Statistical test: are the two models significantly different? —
from scipy import stats
t_stat, p_val = stats.ttest_rel(rf_scores, nb_scores)
print(f'\nPaired t-test RF vs NB: t={t_stat:.3f}, p={p_val:.4f}')

# — 4. k-Means Clustering —————
inertias = []
K_range = range(1, 9)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(K_range, inertias, 'bo-', lw=2, markersize=8)
plt.axvline(3, color='red', linestyle='--', label='True k=3 (Elbow)')
plt.xlabel('Number of Clusters k'); plt.ylabel('Inertia (Within-cluster SS)')
plt.title('Elbow Method for Optimal k'); plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

```
# — 5. Confusion Matrix —————
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X_scaled, y, test_size=0.25, stratify=y,
random_state=42)
rf.fit(X_tr, y_tr)
cm = confusion_matrix(y_te, rf.predict(X_te))

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Random Forest Confusion Matrix'); plt.xlabel('Predicted');
plt.ylabel('Actual')
plt.tight_layout(); plt.show()
```

Mini Quiz

Q1: Cross-validation gives accuracy scores [0.92, 0.88, 0.91, 0.93, 0.89]. What is the mean and std, and what do they tell you?

✓ *Answer: Mean = 0.906 (90.6% avg accuracy). Std = 0.019. Low std means the model is consistent across different data splits — a good sign of robustness, not just luck on one split.*

Q2: In k-Means, what does the 'inertia' metric measure, and how does it help choose k?

✓ *Answer: Inertia = total within-cluster sum of squared distances. Lower = tighter clusters. Plot inertia vs k (elbow plot): it decreases as k increases, but flattens after the true optimal k. The 'elbow' point is the best k.*

Chapter 14: Complete Mini Project — End-to-End Data Analysis

Project: Predicting Diabetes Risk from Health Metrics

This project brings together all the statistical techniques you have learned into a complete data analysis pipeline. We will load the Pima Indians Diabetes dataset, clean it, explore it, find patterns, and build a predictive model — using statistics at every step.

Project Overview

Dataset: Pima Indians Diabetes (via sklearn or seaborn)

Target: Predict whether a patient has diabetes (binary: 0=No, 1=Yes)

Features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age

Steps:

1. Load and inspect data
2. Identify and handle missing values (many are encoded as 0)
3. Descriptive statistics
4. Distribution visualisation
5. Outlier detection
6. Correlation analysis
7. Feature scaling
8. Model training and evaluation
9. Statistical interpretation of results

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve)
import warnings; warnings.filterwarnings('ignore')

print('=' * 60)
```

```
print(' DIABETES PREDICTION: COMPLETE STATISTICAL ANALYSIS')
print('=' * 60)

# — STEP 1: LOAD DATA —————
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-
diabetes.data.csv'
cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
        'Insulin', 'BMI', 'DPF', 'Age', 'Outcome']
try:
    df = pd.read_csv(url, header=None, names=cols)
except:
    # Fallback: use sklearn's load_diabetes (different, continuous target)
    from sklearn.datasets import load_breast_cancer
    data = load_breast_cancer(as_frame=True)
    df = data.frame.copy()
    df.columns = list(data.feature_names) + ['Outcome']
    df = df[['mean radius', 'mean texture', 'mean perimeter', 'mean area',
            'mean smoothness', 'Outcome']].copy()
    cols = df.columns.tolist()

print(f'\nDataset shape: {df.shape}')
print(f'Columns: {df.columns.tolist()}')
print(f'\nFirst 5 rows:')
print(df.head())

# — STEP 2: DESCRIPTIVE STATISTICS —————
print('\n' + '-'*50)
print('STEP 2: DESCRIPTIVE STATISTICS')
print('-'*50)
print(df.describe().round(2))

# Target distribution
print('\nTarget class distribution:')
print(df['Outcome'].value_counts())
print(df['Outcome'].value_counts(normalize=True).round(3))

# — STEP 3: MISSING VALUES —————
print('\n' + '-'*50)
print('STEP 3: MISSING VALUE ANALYSIS')
print('-'*50)
# In Pima dataset, 0 means missing for medical readings
zero_check_cols = [c for c in cols if c not in ['Pregnancies', 'Outcome']]
for col in zero_check_cols:
    if col in df.columns:
        n_zero = (df[col] == 0).sum()
        pct = n_zero / len(df) * 100
        if n_zero > 0: print(f' {col:25s}: {n_zero} zeros ({pct:.1f}%) - likely
missing')

# Replace 0s with NaN for medical columns, then impute with median
medical_cols = [c for c in zero_check_cols if c in df.columns]
df_clean = df.copy()
for col in medical_cols:
    df_clean[col] = df_clean[col].replace(0, np.nan)
    df_clean[col].fillna(df_clean[col].median(), inplace=True)
print('Zeros replaced with median imputation.')

# — STEP 4: DISTRIBUTION ANALYSIS —————
print('\n' + '-'*50)
print('STEP 4: DISTRIBUTION AND SKEWNESS ANALYSIS')
```

```

print('-'*50)

numeric_features = [c for c in df_clean.columns if c != 'Outcome'][:6]

for col in numeric_features:
    sk = stats.skew(df_clean[col])
    ku = stats.kurtosis(df_clean[col])
    print(f' {col:30s}: skew={sk:+.3f} kurtosis={ku:+.3f}')

# Plot distributions
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for ax, col in zip(axes.flat, numeric_features):
    for label, grp in df_clean.groupby('Outcome'):
        name = 'Diabetes' if label == 1 else 'No Diabetes'
        ax.hist(grp[col], bins=25, alpha=0.6, label=name, edgecolor='white')
        ax.set_title(col); ax.legend(fontsize=8); ax.grid(axis='y', alpha=0.3)
plt.suptitle('Feature Distributions by Diabetes Status', fontsize=13, y=1.01)
plt.tight_layout(); plt.show()

# — STEP 5: OUTLIER DETECTION —————
print('\n' + '-'*50)
print('STEP 5: OUTLIER DETECTION (IQR Method)')
print('-'*50)
df_no_outliers = df_clean.copy()
for col in numeric_features:
    Q1, Q3 = df_clean[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    n_out = ((df_clean[col] < lo) | (df_clean[col] > hi)).sum()
    print(f' {col:30s}: {n_out} outliers (bounds: [{lo:.1f}, {hi:.1f}])')

# — STEP 6: CORRELATION ANALYSIS —————
print('\n' + '-'*50)
print('STEP 6: CORRELATION ANALYSIS')
print('-'*50)
corr = df_clean.corr(numeric_only=True)
target_corr = corr['Outcome'].drop('Outcome').sort_values(ascending=False)
print('Correlation with Outcome (Diabetes):')
print(target_corr.round(3))

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlBu', center=0,
            linewidths=0.5, square=True, cbar_kws={'shrink':0.75})
plt.title('Correlation Matrix - Diabetes Dataset', fontsize=13)
plt.tight_layout(); plt.show()

# — STEP 7: STATISTICAL TESTS —————
print('\n' + '-'*50)
print('STEP 7: HYPOTHESIS TESTS (t-tests per feature)')
print('-'*50)
diabetic = df_clean[df_clean['Outcome']==1]
non_diabetic = df_clean[df_clean['Outcome']==0]
for col in numeric_features:
    t, p = stats.ttest_ind(diabetic[col], non_diabetic[col])
    sig = '*** SIGNIFICANT' if p < 0.05 else ''
    print(f' {col:30s}: p={p:.5f} {sig}')

# — STEP 8: MODEL TRAINING AND EVALUATION —————
print('\n' + '-'*50)

```

```

print('STEP 8: MODEL TRAINING AND EVALUATION')
print('-'*50)

X = df_clean[numeric_features].values
y = df_clean['Outcome'].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(100, random_state=42),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
best_model, best_auc = None, 0

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train_s, y_train, cv=cv,
    scoring='roc_auc')
    print(f'\n{name}')
    print(f' CV AUC: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}')
    model.fit(X_train_s, y_train)
    y_pred = model.predict(X_test_s)
    y_prob = model.predict_proba(X_test_s)[:,1]
    test_auc = roc_auc_score(y_test, y_prob)
    print(f' Test AUC: {test_auc:.3f}')
    print(classification_report(y_test, y_pred, target_names=['No
    Diabetes', 'Diabetes']))
    if test_auc > best_auc: best_auc, best_model = test_auc, (name, model, y_prob)

# — STEP 9: ROC CURVE —————
plt.figure(figsize=(7, 5))
for name, model in models.items():
    y_p = model.predict_proba(X_test_s)[:,1]
    fpr, tpr, _ = roc_curve(y_test, y_p)
    auc = roc_auc_score(y_test, y_p)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC={auc:.3f})')
plt.plot([0,1],[0,1], 'k--', lw=1, label='Random Classifier')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curves – Diabetes Prediction'); plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()

# — STEP 10: FEATURE IMPORTANCE —————
rf_model = models['Random Forest']
importances = pd.Series(rf_model.feature_importances_, index=numeric_features)
importances_sorted = importances.sort_values(ascending=True)

plt.figure(figsize=(8, 4))
importances_sorted.plot(kind='barh', color='steelblue', edgecolor='white')
plt.title('Feature Importances – Random Forest'); plt.xlabel('Importance')
plt.tight_layout(); plt.show()

print('\n' + '='*60)
print('PROJECT COMPLETE!')

```

```
print('='*60)
```

Chapter 15: Summary and Further Reading

15.1 The Statistics-ML Connection: Full Picture

Statistics Topic	Core Idea	ML Application
Data Types	NOIR scale, categorical vs numeric	Encoding, preprocessing choices
Central Tendency	Mean, median, mode	Feature summary, robust loss functions
Dispersion	Std, variance, IQR	Normalisation, anomaly detection
Normal Distribution	Bell curve, μ and σ	Weight init, Gaussian assumptions
Skewness/Kurtosis	Shape of distribution	Feature transformations, outlier risk
Outliers	IQR and Z-score methods	Data cleaning, robust models
Scaling	Min-max, standardisation	Required for distance-based/gradient models
Correlation	Pearson r , direction and strength	Feature selection, multicollinearity
Covariance	Joint variability	PCA, covariance matrix in ML
Hypothesis Testing	p-value, H_0 vs H_1	A/B testing, model comparison
Linear Regression	Line of best fit, R^2 , RMSE	Prediction, interpretability
EDA	Explore before modelling	Data quality, pattern discovery
Bias-Variance	Underfitting vs overfitting	The central ML challenge

15.2 Recommended Next Steps

Courses and Books

- 'Statistics for Data Science' — Udemy/Coursera practical courses
- 'Naked Statistics' by Charles Wheelan — intuitive, non-technical overview
- 'An Introduction to Statistical Learning' (ISLR) — free PDF, R/Python code, gold standard
- 'Think Stats' by Allen Downey — Python-first statistics, free online
- 'Practical Statistics for Data Scientists' — O'Reilly, applied focus

Practice Projects

7. Load the Titanic dataset. Do full EDA: distributions, correlations, missing values. Build a classifier.
8. Load any economic dataset. Find skewed features, apply transformations, check if it improves regression R^2 .
9. Simulate the Central Limit Theorem: take samples from a skewed distribution, plot the distribution of sample means.

10. Build an outlier detector using both Z-score and IQR methods. Compare what they flag on a real dataset.
11. Compare two ML models using paired t-test on their cross-validation scores. Determine if the difference is statistically significant.

15.3 Final Comprehensive Quiz

Mini Quiz

Q1: What does a p-value of 0.02 mean in plain English?

✓ *Answer: If the null hypothesis were true, there is only a 2% chance of observing data this extreme (or more extreme). Since $2\% < 5\%$, we reject H_0 and conclude there is a statistically significant effect.*

Q2: A feature has skewness = 2.1. What does this tell you and what should you do?

✓ *Answer: Right-skewed distribution with a long tail to the right. Most values cluster low with a few very high values. Apply log transform or Box-Cox transform before using it in linear/logistic regression.*

Q3: Training $R^2 = 0.98$, Test $R^2 = 0.51$. Diagnose the problem.

✓ *Answer: Severe overfitting (high variance). The model memorised training noise. Fix: regularise (Ridge/Lasso), add more training data, use cross-validation, simplify the model, apply dropout if neural network.*

Q4: Feature A correlates 0.91 with Feature B. Should you keep both?

✓ *Answer: Generally no — this is multicollinearity. Both carry the same information, making regression coefficients unreliable. Options: drop one, compute PCA, or use Ridge/Lasso which handle this automatically.*

Q5: How does standardisation (z-score scaling) help gradient descent training?

✓ *Answer: It makes the loss surface more 'circular' rather than elongated. Without scaling, features with large ranges dominate gradients, causing erratic zig-zagging. With equal scales, gradient steps are balanced and converge much faster.*

End of Statistics for Machine Learning, Deep Learning, AI & Data Science

Data is not information. Information is not knowledge. But with statistics, data becomes insight.